
C-programmering

Pekare

Minneslayout och adresser

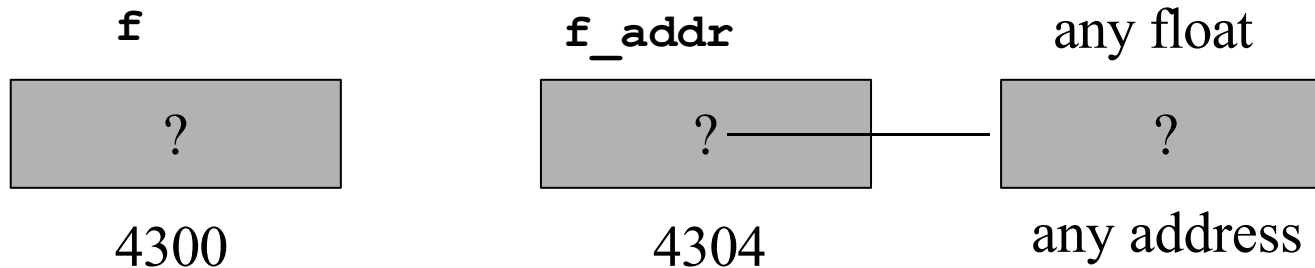
```
int x = 5, y = 10;  
float f = 12.5, g = 9.8;  
char c = 'c', d = 'd';
```

5	10	12.5	9.8	c	d
4300	4304	4308	4312	4316	4317

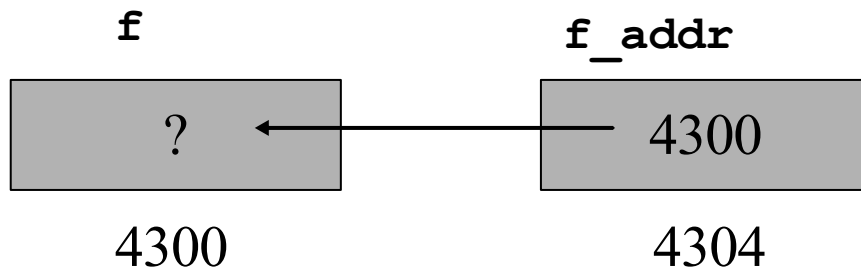
Pekare

- Pekare* = variabel som innehåller adress till annan variabel

```
float f;          /* datavariabel */  
float *f_addr;    /* pekarvariabel */
```



```
f_addr = &f; /* & = adressoperator */
```



Exempel på pekare

```
#include <stdio.h>

void main(void) {
    int j;          /* j = <definierad> */
    int *ptr;       /* ptr = <definierad> */

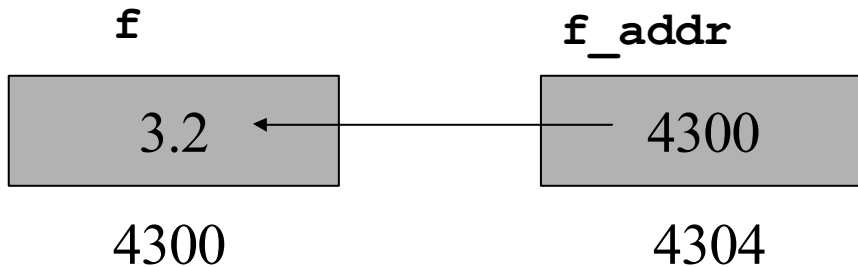
    ptr=&j;         /* initialisera före användning */
                  /* *ptr=4 initialiserar ej ptr */

    *ptr=4;         /* j <- 4 */

    j=*ptr;         /* j <- ??? */
}
```

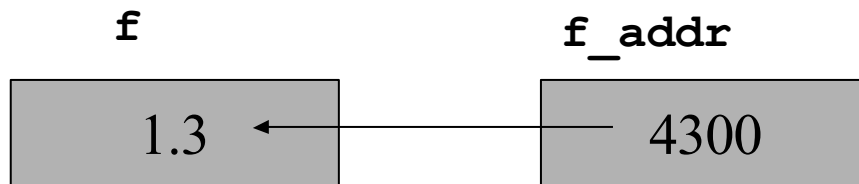
Pekare 2

```
*f_addr = 3.2; /* indirekt adressering */
```



```
float g=*f_addr; /* indirektion:g är nu 3.2 */
```

```
f = 1.3;
```



4300

4304

Pekare – byte mellan datatyper

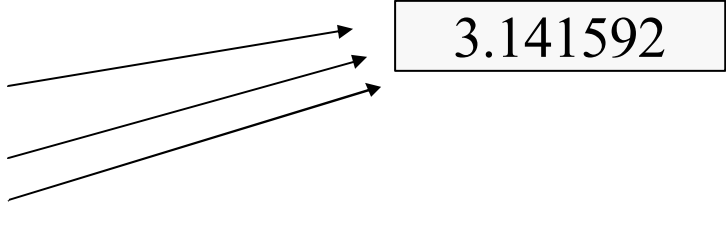
```
#include <stdio.h>
```

```
int main(void) {  
    float f = 3.141592;  
    void *ptr; /* allmän pekare, ej till typ */  
    float *f_ptr;  
    int *i_ptr;
```

```
    ptr = (void *)&f;
```

```
    f_ptr = &f;
```

```
    i_ptr = (int *)&f
```



3.141592

```
    printf("Val: %f, adress: %p, : int %i\n",  
           *f_ptr, ptr, *i_ptr);
```

```
}  
% Val: 3.141592, adress: 0xbf82e238, int: 1065353216
```

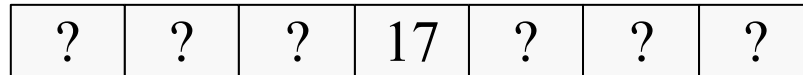
sizeof()-operatörn

- `sizeof(x)` – ger storleken för datatypen `x`
- t.ex.
- `sizeof(int) = 4`
- `sizeof(float) = 4`
- `sizeof(int *) = 4`
- `sizeof(Person) = 40`

`char[30] + int + int + alignment`

Vad är en räkka ?

```
int main(void) {  
    int lottorad[7];  
  
    lottorad[3] = 17;  
}
```



4300

variabeln `lottorad` är de fakto en pekare till det första elementet i räckan
`lottorad[3]` kan räknas som
`lottorad + 3 * sizeof(int)`

Mer om adressering av räckor

Kompilatorn gör följande beräkningar

```
&lottorad[3] = lottorad + sizeof(int)*3
```

Man kan också accessera räckan såhär:

```
*(lottorad+3) = 17 /* ekvivalent med lottorad[3]=17 */
```

Dock:

```
*lottorad+3 = <int>
```

Vi kan också accessera räckan via generell pekare

```
int *lotto_ptr = lottorad;  
lotto_ptr[3] = 17;
```

Mer om räckor

Vad är då skillnaden mellan

```
int lottorad[7];  
    OCH  
int *lotto_ptr;
```

lottorad : Konstant pekare till en räck, samtidigt som utrymme reserveras)

lotto_ptr: Är en variabel pekare (vi kan ändra värdet på själva pekaren)

Mera pekare

```
int month[12]; /* month is a pointer to base address 430 */
```

```
month[3] = 7;  /* month address + 3 * int elements  
=> int at address (430+3*4) is now 7 */
```

```
ptr = month + 2; /* ptr points to month[2],  
=> ptr is now (430+2 * int elements)= 438 */
```

```
ptr[5] = 12;  
/* ptr address + 5 int elements  
=> int at address (438+5*4) is now 12.  
Thus, month[7] is now 12 */
```

```
ptr++;          /* ptr <- 438 + 1 * size of int = 442 */  
(ptr + 4)[2] = 12; /* accessing ptr[6] i.e., month[9] */
```

- Nu är month[6], *(month+6), (month+4)[2],

~~ptr[3], *(ptr+3) samma heltalsvariabel !!~~

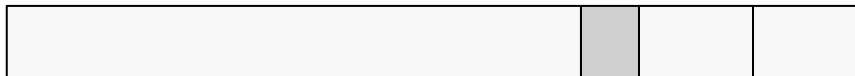
Pekare till strukturer

```
typedef struct t_person {  
    char namn[30];  
    int alder;  
    int skonummer;  
} Person;
```

```
typedef Person * pPerson;
```

```
int main() {  
    Person pers;  
    pPerson ptr_pers = &pers;  
}
```

Padding till hela 4 byte



namn
(30)

alder skonummer
(4) (4)

Pekare till pekare

- Ibland behövs dubbel indirektion, t.ex funktion som allokerar en ny datastruktur

```
int getnewperson(Person **dp) {
    *dp = (Person *)malloc(sizeof(Person));
    if (*dp) return 0;
    return -1;
}
int main() {
    Person *p;
    getnewperson(&p); /* ge en pekare till pekaren */
    p->alder = 23;
}
```

Vad fel??

```
#include <stdio.h>
```

```
void dosomething(int *ptr);
```

```
main() {  
    int *p;  
    dosomething(p)  
    printf("%d", *p);    /* will this work ? */  
}
```

```
void dosomething(int *ptr){ /* passed and returned by  
                           reference */
```

```
    int temp=32+12;
```

```
    ptr = &(temp);
```

```
}
```

```
/* compiles correctly, but gives run-time error
```

```
Vi returnerar en pekare til stacken, som inte
```

```
är giltig efter att vi återvänder */
```

Programmering i C/C++ / JB

Pekare till funktioner

```
int func(); /*function returning integer*/  
int *func(); /*function returning pointer to integer*/  
int (*func)(); /*pointer to function returning integer*/  
int *(*func)(); /*pointer to func returning ptr to int*/
```

- Fördel ? mera flexibilitet

Pekare till funktion - Exempel

```
#include <stdio.h>

void myproc (int d);
void mycaller(void (* f)(int), int param);

void main(void) {
    myproc(10);          /* call myproc with parameter 10*/
    mycaller(myproc, 10); /* and do the same again ! */
}

void mycaller(void (* f)(int), int param){
    (*f)(param);         /* call function *f with param */
}

void myproc (int d){
    . . .                /* do something with d */
}
```


Mer komplicerat

Deklarera en räkka med N pekare till funktioner som returnerar pekare till funktioner som returnerar pekare till tecken

1. `char *(*a[N])()();`

2. Build the declaration up in stages, using typedefs:

```
typedef char *pc; /* pointer to char */
typedef pc fpc(); /* function returning pointer to char */
typedef fpc *pfpc; /* pointer to above */
typedef pfpc fpfpc(); /* function returning... */
typedef fpfpc *pfpfpc; /* pointer to... */
pfpfpc a[N]; /* array of... */
```

By-value / by-reference

Tidigare sades att ALLA parametrar i C överförs by-value

Hur överförs då t.ex. strängar ??

```
char str1[30] = "Hej", str2[30];  
strcpy(str2, str1);
```

Det som nu överförs är pekarens värde "*by-value*" (str1, str2 är konstanta pekare), detta kunde i princip också kallas *by-reference*

By-value / by-reference

I C har du alltid kontroll över det minne du själv allokerat

→ Ingen funktion du anropar kan ändra ditt minne utan "ditt tillstånd" (dvs för att en funktion skall kunna ändra lokala variabler, måste den få en pekare till denna variabel (by reference))

By value / by reference

```
int inc(int a) { /* denna funktion kan ej direkt
                ändra värdet variabeln */
    return a+1;
}
void inc_ref(int *a) { /* denna funktion får en
                       pekare till variabeln, kan
                       direkt inkrementera
                       variabeln */
    *a++;
}

int main() {
    int b=0;
    b = inc(b);    /* by value: variabelvärdet */
    inc_ref(&b);    /* by value: pekaren till variabeln */
}
```

By value / by reference

```
#include <stdio.h>
void swap(int *, int *);

main() {
    int num1 = 5, num2 = 10;
    swap(&num1, &num2);
    printf("num1 = %d and num2 = %d\n", num1, num2);
}

void swap(int *n1, int *n2) { /* passed and returned by
                                reference */
    int temp;

    temp = *n1;
    *n1 = *n2;
    *n2 = temp;
}
```

Räckor som parametrar

```
void init_array(int array[], int size) ;
```

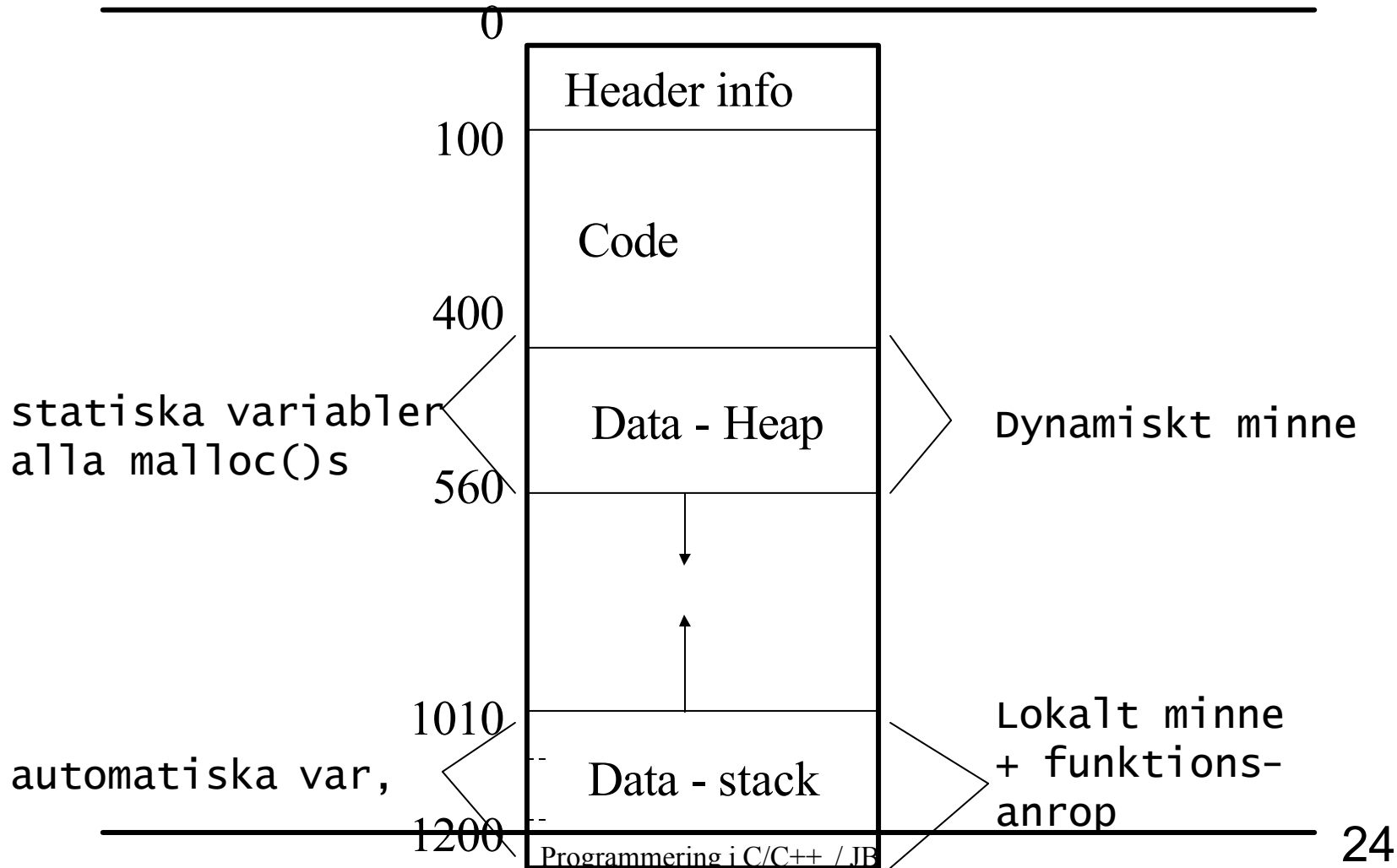
```
int main(void) {  
    int list[5];  
  
    init_array(list, 5);  
    for (i = 0; i < 5; i++)  
        printf("next:%d", array[i]);  
}
```

```
void init_array(int array[], int size) { /* varför  
    storlek ? */  
    /* räckor alltid by-reference = pekaren by-value  
    */  
    int i;  
    for (i = 0; i < size; i++)  
        array[i] = 0;  
}
```

C-programmering

Minneshantering

Minnesrymd för program



Dynamisk minnesallokering

- Explicit allokering och deallokering

```
#include <stdio.h>
```

```
int main(void) {  
    int *ptr;  
        /* allokera utrymme för ett heltal */  
    ptr = malloc(sizeof(int));  
  
        /* används minnesutrymmet */  
    *ptr=4;  
  
    free(ptr);  
        /* frigör det allokerade minnet */  
}
```

Dynamiskt minne - funktioner

```
#include <malloc.h>
void *malloc(size_t size);
void *calloc(size_t nmem, size_t size);
void *free(void *ptr);
void *realloc(void *ptr, size_t size);
```

`malloc` - returnerar pekare till minnesblock av storlek `size`

`realloc` - ökande / minskade av minne, gemensamt om oförändrad

`free` - frigör minne på pekare `ptr`

Dynamisk minnesallokering - exempel

```
#include <stdio.h>
#include <malloc.h>

int main() {
    char *buffer;
    pPerson ptr_pers;

    buffer = (char *)malloc(50 * sizeof(char));
    ptr_pers = (pPerson)malloc(40 * sizeof(Person));

    if (!buffer) return (-1); /* om malloc ej lyckades */
    if (!ptr_pers) return (-1);
    strcpy(buffer, "Hej, här är texten");
    strcpy(ptr_pers[0]->namn, "Axel Eklund");
    ptr_pers[0]->alder = 32;
    return 0;
}
```

OBS!! fält i strukturer adresseras med -> då pekare
till struktur används

Dynamiska matriser

```
#include <stdio.h>
#include <malloc.h>

float ** matrix_alloc(int rows, int cols) {
    float **matr;
    int i;
    /* Allokera pekare till kolumnräckor */
    matr = (float **)malloc(sizeof(float *)*rows);
    for(i=0; i<cols; i++) {
        /* Allokera kolumnräckorna */
        matr[i] = (float *)malloc(sizeof(float)*cols);
    }
    return matr;
}
```

Minnesläckor

- Allokerat minne (malloc(), realloc(), calloc()) har inte ett motsvarande free()
- Med tiden växer den allokerade minnesmängden
 - I serverbruk med tiden katastrofalt
 - För klientprogram (typ webbläsare) främst störande
- Hur kan man undvika
 - Allokering / deallokering på välkontrollerade platser
 - Noggrann felhantering (så att inte free() blir bortglömt p.g.a. ovanligt programflöde)
 - Verktyg vid utvecklingsskedet

Minnesläckor - exempel

```
#include <stdio.h>
#include <malloc.h>

int myfunc() {
    char *buffer;

    buffer = (char *)malloc(50 * sizeof(char));
    if (!buffer) exit(-1); /* om malloc ej lyckades */
    scanf("%d", &newval);
    if (newval<0) return -1; /* error */
    ...
    free(buffer); /* programflödet når inte alltid
hit = möjlig minnesläcka */
}
```

Preprocessor

- Kompilering av C kod föregås av en prokompilator
- Prekompilatorn genererar en ny "fil" med källkod genom att
 - ersätta definierade textsträngar
 - inkludera andra filer
 - konditional inkludering av textavsnitt
- Denna nya enhetliga fil körs genom den egentliga C-kompilatorn

Preprocessor
